

A RL Fuzzy Backoff Mechanism for Task Offloading in VEC 5G NR Environments

Antônio Sérgio de Sousa Vieira^{1,2*}, Alisson Barbosa de Souza^{1,3} and
Joaquim Celestino Júnior¹

^{1*}Intelligent Networks and Systems Group (INETSYS), State University of Ceará, Av.
Dr. Silas Munguba, 1700, Fortaleza, 60714-903, CE, Brazil.

²Federal Institute of Education, Science and Technology, Av. da Universidade, 102,
Itapipoca, 62500-000, CE, Brazil.

³Federal University of Ceará, Av. José F. Queiroz, 5003, Quixadá, 63902-580, CE, Brazil.

*Corresponding author(s). E-mail(s): sergio.vieira@ifce.edu.br;
Contributing authors: alisson@ufc.br; joaquim.celestino@uece.br;

Abstract

Keywords: Task Offloading, Vehicular Edge Computing, Deep Reinforcement Learning, Fuzzy Logic, 5G NR

1 Introduction

A ampla difusão de tecnologias de comunicação, como *5G New Radio* (NR) e *Vehicle-to-Everything* (V2X), impulsionou a sofisticação das aplicações em ambientes veiculares, as quais exigem alto desempenho em tempo real. Veículos modernos possuem múltiplos sensores, unidades de processamento e interfaces de comunicação em suas Unidades de Bordo (*On-Board Units* – OBUs), viabilizando a coleta e o processamento de grandes volumes de dados. Contudo, a crescente complexidade dos serviços e a demanda por processamento massivo de dados frequentemente excedem a capacidade computacional dos dispositivos embarcados.

Uma solução trivial consistiria no aumento da capacidade de hardware das OBUs, possibilitando maior processamento local. No entanto, o atual período de transição tecnológica faz surgir cenários heterogêneos, nos quais coexistem OBUs

com capacidades computacionais diferentes [?] e tecnologias de comunicação distintas [?]. Essa diversidade impõe um desafio fundamental à implementação de aplicações veiculares em larga escala, especialmente no contexto de Cidades Inteligentes (*Smart Cities*) [?].

Nesse ambiente estocástico e altamente dinâmico, muitas aplicações, em particular aquelas voltadas à condução autônoma, impõem requisitos rigorosos de latência [?]. A execução integral do processamento nos Equipamentos de Usuário (*User Equipments* – UEs) frequentemente inviabiliza o cumprimento de tais requisitos, dadas as restrições físicas de energia e capacidade computacional dos *chips* embarcados [?].

Para mitigar esse problema, o paradigma de Computação de Borda Veicular (*Vehicular Edge Computing* – VEC) estabelece-se como uma arquitetura importante. O VEC permite que os UEs utilizem sua infraestrutura robusta disposta na vizinhança das vias para a redução de

latência por meio da transferência da responsabilidade pelo processamento de dados complexos, técnica denominada descarregamento de tarefas (*task offloading*).

Além da mobilidade veicular contribuir bastante na dinamicidade do cenário, os dispositivos de borda operam sob diferentes regimes de processamento e consumo energético e atendem demandas heterogêneas das aplicações, flutuações denominadas *drifts*. Em cenários reais, a disputa por recursos limitados ocorre entre múltiplos UEs. Cada decisão de descarregamento altera o estado global do sistema, induzindo um *drift* temporal no processamento devido ao enfileiramento de tarefas. Quando UEs operam sob políticas egoístas, convergindo simultaneamente para recursos ociosos, ocorre o “efeito manada” (*herd effect*), fenômeno que compromete a estabilidade do sistema e degrada a Qualidade de Serviço (QoS).

Para enfrentar a natureza competitiva desses ambientes, este trabalho propõe um mecanismo descentralizado para a coordenação indireta de janelas de contenção de descarregamento, fundamentado em Lógica *Fuzzy* e Aprendizado por Reforço (*Reinforcement Learning* – RL) em cenários multiagente. O objetivo central é otimizar a alocação de recursos e satisfazer os requisitos de QoS das aplicações, reduzindo o consumo energético global e assegurando o balanceamento de carga, avaliado pelo Índice de Justiça de Jain (*Jain’s Fairness Index*) [?].

As principais contribuições deste trabalho são detalhadas a seguir:

1. **Quantificação Matemática do Congestionamento Sistêmico:** Análise do impacto sistêmico quando múltiplos agentes convergem simultaneamente para a mesma política;
2. **Aprendizado de Campo Médio (*Mean Field Learning*):** Utilização de uma abordagem descentralizada para estimar a densidade local de agentes e mitigar a explosão do espaço de estados;
3. **Mecanismo de *Backoff* Baseado em Lógica *Fuzzy*:** Proposição de um método adaptativo para gerenciar o acesso aos recursos, resiliente às incertezas do ambiente veicular.

2 Related Work

3 System Model

We consider a VEC system operating in a 5G NR environment, consisting of a set of vehicles $\mathcal{V} = \{1, 2, \dots, V\}$ and a set of Roadside Units (RSUs) $\mathcal{R}$. The system evolves over discrete time slots $t \in \{0, 1, \dots, T\}$.

Table 1 Table of Notations

Notation	Description
T_i	Task i , defined as (d_i, c_i)
d_i	Data size of task i (bits)
c_i	Computational density (cycles/bit)
f^l, f^e	CPU frequency of vehicle and edge server
L_i^l, L_i^e	Local and Edge computation latency
$R_{i,j}$	Transmission rate from vehicle i to node j
E_i	Total energy consumption for task i
$Q_j(t)$	Processing queue waiting time at node j
κ	Energy efficiency coefficient

3.1 Task and Communication Model

Each vehicle i generates a task T_i at time t , characterized by the tuple (d_i, c_i) , where d_i denotes the input data size and c_i denotes the computational workload in CPU cycles per bit. The transmission rate $R_{i,j}$ between vehicle i and node j is modeled using the Shannon-Hartley theorem for 5G NR channels:

$$R_{i,j} = B \log_2 \left(1 + \frac{P_{tx} |h_{i,j}|^2}{N_0 + I_{i,j}} \right) \quad (1)$$

where B is the bandwidth, P_{tx} is the transmission power, $h_{i,j}$ is the channel gain, N_0 is the noise spectral density, and $I_{i,j}$ represents interference.

3.2 Latency Model

For local execution, the latency L_i^l is determined by the vehicle’s CPU frequency f_i^l :

$$L_i^l = \frac{d_i c_i}{f_i^l} \quad (2)$$

For offloading to an edge server j , the total latency L_i^e includes transmission time, queueing

delay $Q_j(t)$, and processing time:

$$L_i^e = \frac{d_i}{R_{i,j}} + Q_j(t) + \frac{d_i c_i}{f_j^e} \quad (3)$$

3.3 Energy Model

The local energy consumption is modeled as:

$$E_i^l = \kappa (f_i^l)^2 d_i c_i \quad (4)$$

where κ is the effective switched capacitance coefficient.

For offloading, the energy consumption considers the transmission power:

$$E_i^{off} = P_{tx} \frac{d_i}{R_{i,j}} + E_{tail} \quad (5)$$

3.4 Non-Stationary Workload Model

Standard simulation models often assume stationary arrival processes (e.g., Poisson) and constant environmental parameters. However, real-world vehicular networks exhibit non-stationary behaviors, such as traffic bursts and sudden changes in channel quality due to mobility [? ?]. To evaluate our solution under realistic conditions, we model the environment using a regime-switching process $Z(t)$, where system parameters (e.g., task arrival rate λ , CPU load) evolve through distinct phases (regimes).

Figure 1 illustrates a trajectory of regimes over time. This approach allows us to test the adaptability of the offloading policy to abrupt changes in the environment.

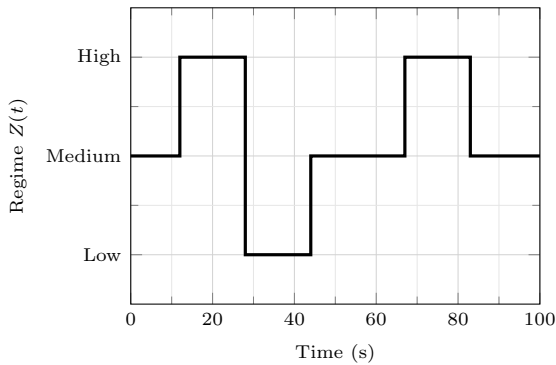


Fig. 1 Example of a regime-switching trajectory $Z(t)$ representing non-stationary environmental changes (e.g., traffic load levels).

Furthermore, to model the physical inertia of system responses (e.g., queue buildup or thermal throttling), we map the discrete regime target $u(t)$ to an observable variable $y(t)$ using a first-order dynamic equation:

$$y_{k+1} = \alpha y_k + (1 - \alpha) u_k \quad (6)$$

where $\alpha \in [0, 1)$ is the inertia factor. Figure 2 shows how this results in smooth transitions, mimicking realistic metrics.

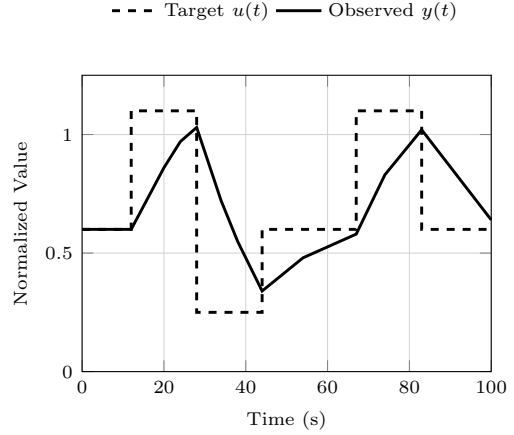


Fig. 2 System response $y(t)$ with inertia tracking the regime target $u(t)$, avoiding unrealistic instantaneous jumps.

4 Proposed Solution

4.1 Fuzzy Logic

4.1.1 Fuzzification

4.1.2 Fuzzy Rules and Inference

4.2 Deep Reinforcement Learning Agent

4.2.1 State Space (S_t)

4.2.2 Action Space (A_t)

4.2.3 Reward Function (R_t)

5 Experimental Evaluation

5.1 Simulation Setup

6 Conclusion